EventBliss — Event Management System

Full-Stack Web Application | Project Submission

Table of Contents

[Project Overview](#1-project-overview)
[Tech Stack](#2-tech-stack)
[Database Design](#3-database-design)
[Entity-Relationship Diagram](#4-entity-relationship-diagram)
[SQL Queries Used](#5-sql-queries-used)
[Application Features](#6-application-features)
[Authentication & Security](#7-authentication--security)
[Email & Notification System](#8-email--notification-system)
[User Flows](#9-user-flows)
[Project Structure](#10-project-structure)
[Challenges & Learning](#11-challenges--learning)

1. Project Overview

EventBliss is a full-stack web-based Event Management System built for a college environment. It allows students to browse, register for, and track events, while giving admins complete tools to create and manage events, venues, registrations, attendance, and sub-admin permissions — all backed by a relational MySQL database.

Core Goals:

- Demonstrate real-world database design with multiple related tables and foreign keys
- Implement secure, multi-factor authentication using JWT tokens, OTP verification, and Google OAuth
- Show practical use of SQL: INSERT, SELECT with JOINs, UPDATE, DELETE, COUNT, GROUP BY, ON DUPLICATE KEY UPDATE
- Apply role-based access control at both the database and application level
- Build a complete email notification pipeline with automated reminders

2. Tech Stack

Layer	Technology
Backend	Python 3, Flask 3.0
Database	MySQL (via mysql-connector-python)

Authentication	JWT (PyJWT 2.8), Google OAuth 2.0 (Authlib 1.3)
Email	Flask-Mail (Gmail SMTP, TLS on port 587)
Scheduling	APScheduler 3.10 (BackgroundScheduler)
Frontend	HTML5, CSS3 (custom glass-card UI), Jinja2 templates, JavaScript (fetch API)
Environment	python-dotenv (.env config), Gunicorn (production WSGI)

3. Database Design

The system uses 7 tables with well-defined relationships.

students

Stores all registered student accounts. Supports both email/password and Google OAuth login.

Column	Type	Constraints
student_id	INT	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(100)	NOT NULL
username	VARCHAR(100)	UNIQUE, NULL (optional)
email	VARCHAR(200)	UNIQUE, NOT NULL
password	VARCHAR(255)	NOT NULL (GOOGLE_OAUTH for OAuth users)
phone	VARCHAR(20)	NULL
year	VARCHAR(20)	NULL
dept	VARCHAR(100)	NULL
section	VARCHAR(20)	NULL
roll_no	VARCHAR(50)	NULL
bio	TEXT	NULL

admins

Stores admin accounts. The default admin is seeded on first startup. Sub-admins are created via the Permissions page.

Column	Type	Constraints
admin_id	INT	PRIMARY KEY, AUTO_INCREMENT
username	VARCHAR(100)	UNIQUE, NOT NULL
email	VARCHAR(200)	UNIQUE, NOT NULL
password	VARCHAR(255)	NOT NULL

events

Stores all events created by admins.

Column	Type	Constraints
event_id	INT	PRIMARY KEY, AUTO_INCREMENT
event_name	VARCHAR(200)	NOT NULL
event_date	DATE	
event_time	TIME	
venue	VARCHAR(200)	(legacy text field)
venue_id	INT	FOREIGN KEY → venues(venue_id)
max_seats	INT	
theme	VARCHAR(200)	
requirements	TEXT	
organiser	VARCHAR(200)	

venues

Stores venue information linked to events. Admins can check real-time availability when creating events.

Column	Type	Constraints
venue_id	INT	PRIMARY KEY, AUTO_INCREMENT
venue_name	VARCHAR(200)	NOT NULL
location	VARCHAR(200)	
capacity	INT	

registrations

Junction table linking students to events (many-to-many). Stores each student's academic details at the time of registration.

Column	Type	Constraints
reg_id	INT	PRIMARY KEY, AUTO_INCREMENT
student_id	INT	FOREIGN KEY → students(student_id)
event_id	INT	FOREIGN KEY → events(event_id)
first_name	VARCHAR(100)	
middle_name	VARCHAR(100)	
last_name	VARCHAR(100)	
year	VARCHAR(10)	

dept	VARCHAR(100)	
section	VARCHAR(10)	
roll_no	VARCHAR(50)	
phone	VARCHAR(20)	
notification_email	VARCHAR(200)	NULL (optional alternate email for reminders)
registration_date	DATETIME	DEFAULT CURRENT_TIMESTAMP
reminder_sent	TINYINT(1)	DEFAULT 0 (tracks if 24-hour reminder was sent)

attendance

Tracks per-student attendance for each event. Populated by the admin on event day.

Column	Type	Constraints
attendance_id	INT	PRIMARY KEY, AUTO_INCREMENT
event_id	INT	NOT NULL
registration_id	INT	NOT NULL
is_present	TINYINT(1)	DEFAULT 0
marked_at	TIMESTAMP	NULL
		UNIQUE KEY (event_id, registration_id)

permissions

Stores fine-grained permissions for each sub-admin username.

Column	Type	Constraints
permission_id	INT	PRIMARY KEY, AUTO_INCREMENT
admin_username	VARCHAR(100)	UNIQUE
can_create_event	BOOLEAN	
can_delete_event	BOOLEAN	
can_edit_event	BOOLEAN	
can_view_registrations	BOOLEAN	

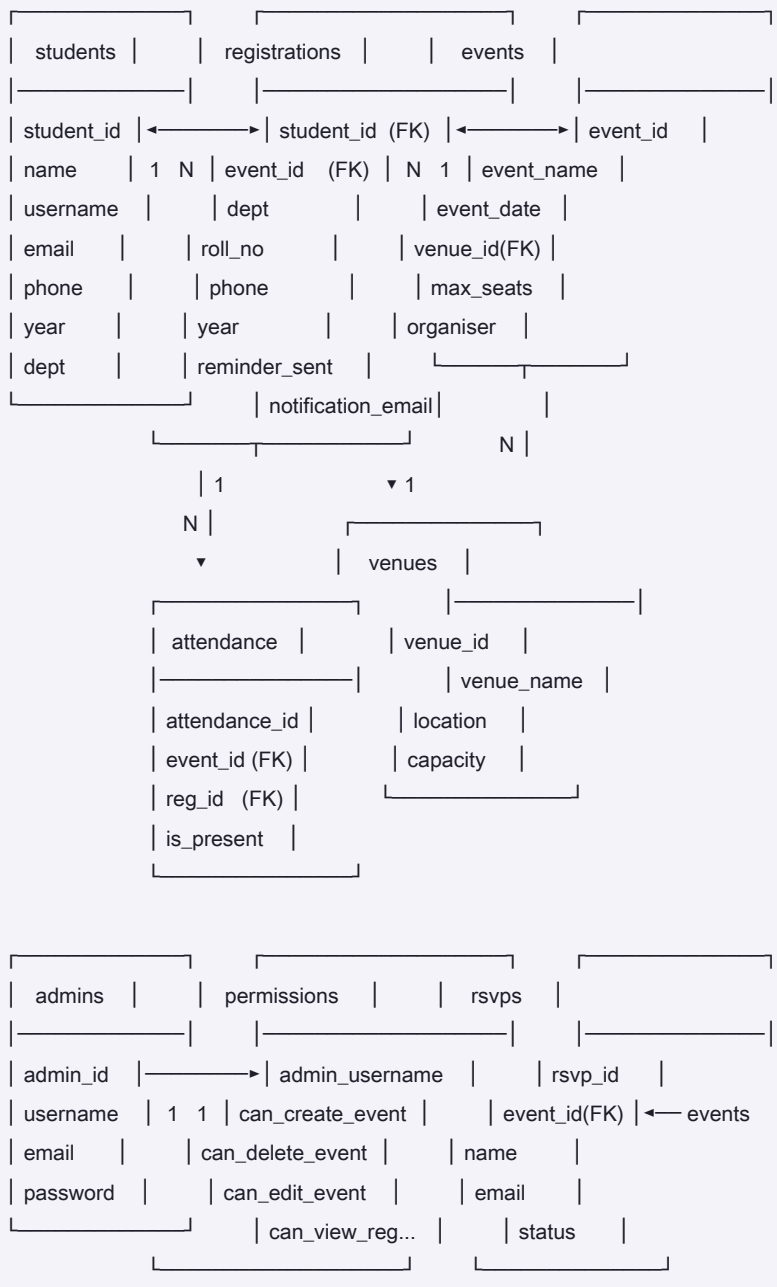
rsvps

Stores RSVPs from external guests (non-student attendees).

Column	Type	Constraints
rsvp_id	INT	PRIMARY KEY, AUTO_INCREMENT

event_id	INT	FOREIGN KEY → events(event_id)
name	VARCHAR(200)	NOT NULL
email	VARCHAR(200)	NOT NULL
phone	VARCHAR(20)	
status	ENUM	'attending', 'not attending', 'maybe'
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP

4. Entity-Relationship Diagram



Relationships:

- students ↔ events : Many-to-Many (through registrations)
- registrations → attendance : One-to-One per event (each registration has one attendance record)
- events → venues : Many-to-One
- admins → permissions : One-to-One
- events → rsvps : One-to-Many

5. SQL Queries Used

CREATE TABLE (Auto-run on startup)

```
CREATE TABLE IF NOT EXISTS admins (
    admin_id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(100) UNIQUE NOT NULL,
    email    VARCHAR(200) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL
);

CREATE TABLE IF NOT EXISTS attendance (
    attendance_id  INT AUTO_INCREMENT PRIMARY KEY,
    event_id       INT NOT NULL,
    registration_id INT NOT NULL,
    is_present     TINYINT(1) DEFAULT 0,
    marked_at      TIMESTAMP NULL,
    UNIQUE KEY uq_attendance (event_id, registration_id)
);
```

INSERT

```
-- Register a student
INSERT INTO students (name, username, email, password) VALUES (%s, %s, %s, %s);

-- Register for an event
INSERT INTO registrations
  (student_id, event_id, first_name, middle_name, last_name,
   year, dept, section, roll_no, phone, notification_email, registration_date)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, NOW());

-- Add a venue
INSERT INTO venues (venue_name, location, capacity) VALUES (%s, %s, %s);
```

SELECT with JOIN

```
-- Events with venue name (COALESCE for legacy data)
SELECT events.event_id, events.event_name, events.event_date,
       COALESCE(venues.venue_name, events.venue) AS venue_display,
       events.max_seats, events.organiser
FROM events
LEFT JOIN venues ON events.venue_id = venues.venue_id
WHERE events.event_date >= CURDATE()
ORDER BY events.event_date ASC, events.event_time ASC;

-- Registrations for an event (student + attendance status)
SELECT r.reg_id,
       CONCAT(r.first_name, ' ', COALESCE(r.middle_name,''), ' ', r.last_name) AS full_name,
       r.roll_no, r.dept, r.section, r.year,
       COALESCE(a.is_present, 0) AS is_present
FROM registrations r
LEFT JOIN attendance a ON r.reg_id = a.registration_id AND a.event_id = %s
WHERE r.event_id = %s
ORDER BY r.roll_no;

-- Student's registered events with attendance status
SELECT events.event_name, events.event_date, events.event_time, events.venue,
       a.is_present
FROM registrations
JOIN events ON registrations.event_id = events.event_id
LEFT JOIN attendance a ON a.registration_id = registrations.reg_id
                      AND a.event_id = events.event_id
WHERE registrations.student_id = %s
ORDER BY events.event_date DESC;
```

SELECT with COUNT and GROUP BY (Analytics)

```
-- Top events by registration count
SELECT e.event_name, COUNT(r.reg_id) AS cnt
FROM events e
LEFT JOIN registrations r ON e.event_id = r.event_id
GROUP BY e.event_id, e.event_name
ORDER BY cnt DESC LIMIT 8;

-- Department breakdown
SELECT dept, COUNT(*) AS cnt
FROM registrations
WHERE dept IS NOT NULL AND dept != ''
GROUP BY dept ORDER BY cnt DESC;

-- Monthly registration trend
SELECT DATE_FORMAT(registration_date, '%Y-%m') AS ym, COUNT(*) AS cnt
FROM registrations
WHERE registration_date IS NOT NULL
GROUP BY DATE_FORMAT(registration_date, '%Y-%m')
ORDER BY ym DESC LIMIT 12;
```

INSERT ... ON DUPLICATE KEY UPDATE (Upsert)

```
-- Mark attendance (insert or update)
INSERT INTO attendance (event_id, registration_id, is_present, marked_at)
VALUES (%s, %s, %s, NOW())
ON DUPLICATE KEY UPDATE is_present=%s, marked_at=NOW();

-- Set or update admin permissions
INSERT INTO permissions
  (admin_username, can_create_event, can_delete_event, can_edit_event, can_view_registrations)
VALUES (%s, %s, %s, %s, %s)
ON DUPLICATE KEY UPDATE
  can_create_event=%s, can_delete_event=%s,
  can_edit_event=%s, can_view_registrations=%s;
```

UPDATE

```
UPDATE students SET password=%s WHERE student_id=%s;
UPDATE students SET name=%s, year=%s, dept=%s, section=%s, roll_no=%s, bio=%s WHERE student_id=%s;
UPDATE students SET email=%s WHERE student_id=%s;
UPDATE students SET phone=%s WHERE student_id=%s;
UPDATE events SET event_name=%s, event_date=%s, venue=%s, max_seats=%s WHERE event_id=%s;
UPDATE venues SET venue_name=%s, location=%s, capacity=%s WHERE venue_id=%s;
UPDATE registrations SET reminder_sent = 1 WHERE reg_id = %s;
```

DELETE

```
DELETE FROM registrations WHERE event_id=%s;
DELETE FROM events WHERE event_id=%s;
DELETE FROM venues WHERE venue_id=%s;
DELETE FROM permissions WHERE admin_username=%s;
```

ALTER TABLE (Incremental schema migration on startup)

```
ALTER TABLE registrations ADD COLUMN registration_date DATETIME DEFAULT CURRENT_TIMESTAMP;
ALTER TABLE registrations ADD COLUMN reminder_sent TINYINT(1) DEFAULT 0;
ALTER TABLE registrations ADD COLUMN notification_email VARCHAR(200) NULL;
ALTER TABLE students ADD COLUMN username VARCHAR(100) UNIQUE NULL;
ALTER TABLE students ADD COLUMN phone VARCHAR(20) NULL;
-- (and year, dept, section, roll_no, bio)
```

6. Application Features

Student Side

Feature	Description
Register	Create account with name, username, email, and password — requires OTP email
Login	Username or email + password login
Google OAuth Login	Sign in with Google — triggers a 6-digit OTP email verification before granting acce
Forgot Password	Enter email → receive 6-digit OTP → verify → set new password (with strength valida
Browse Events	View all upcoming events with seat counts and availability
Register for Event	Fill detailed form (full name, dept, roll no, phone, year, section); optionally add a sep
My Events	View all personal registered events; upcoming events show date/venue; past events

Student Profile	View and edit name, year, dept, section, roll no, bio; change email or phone via OTP

Admin Side

Feature	Description
Admin Login	Username + password, or email-based OTP login (alternative)
Forgot Password	OTP-based reset for admin accounts
Dashboard	Live stats: total events, students, registrations, venues, RSVPs
Analytics Dashboard	Charts for top events by registrations, department breakdown, year-wise distribution
Create Event	Full form with venue selector and real-time venue availability checker (morning/after
Edit Event	Update event details
Delete Event	Removes event and all its registrations
View Registrations	See all students registered for an event
Attendance Tracker	Mark present/absent for each registered student for completed events; shows live p
Manage Venues	Add, edit, delete venues with name, location, and capacity
Guest RSVPs	View RSVPs from non-student attendees per event
Manage Permissions	Register new sub-admins (credentials emailed automatically), grant/revoke create/e

7. Authentication & Security

JWT Cookies

All sessions are managed via signed JWT cookies instead of server-side sessions.

```
Token payload example:
{
  "user_id": 42,
  "user_name": "Rushitha",
  "role": "student",
  "exp": <24 hours from now>
}
```

- Stored as HttpOnly cookies (not accessible via JavaScript)
- Signed with JWT_SECRET using HS256 algorithm
- SameSite=Lax prevents CSRF attacks
- Every protected route calls get_current_user() to verify the token

OTP-Based Verification (All sensitive flows)

Flow	OTP sent to	Expiry
New student registration	New account email	10 minutes
Google OAuth login	Google account email	10 minutes
Student forgot password	Account email	10 minutes
Admin OTP login	Admin email	10 minutes
Admin forgot password	Admin email	10 minutes
Change account email	New email address	10 minutes
Change phone number	Current account email	10 minutes
Add notification email	Notification email address	10 minutes

Password Strength Validation

All passwords (student and admin) must meet:

- At least 8 characters
- At least one uppercase letter
- At least one lowercase letter
- At least one number
- At least one special character

Role-Based Access Control

- Every admin route checks user.get("role") != "admin" before serving
- Sub-admin permissions (create/edit/delete/view) are stored in the DB and checked per action
- abort(403) is returned when a permission is denied
- abort(409) is returned if a student tries to register for the same event twice

8. Email & Notification System

EventBliss uses Flask-Mail with Gmail SMTP to send transactional emails.

Emails Sent

Trigger	Recipient	Content
Student registration	New student	Email verification OTP
Google OAuth login	Student	Google login verification OTP
Student forgot password	Student	Password reset OTP
Admin OTP login	Admin	Login OTP
Admin forgot password	Admin	Password reset OTP
Event registration	Student	Registration confirmation with event details
Sub-admin creation	New sub-admin	Account credentials + permissions

Change email	New email	Verification OTP
Change phone	Current email	Verification OTP
Add notification email	Notification email	Verification OTP
24-hour reminder	Student (or notification email)	Event reminder with details
Reminder batch	Main admin	Summary of reminders sent

Automated Reminder Scheduler

A BackgroundScheduler (APScheduler) runs every hour and emails all students whose event starts within the next 24 hours, where reminder_sent = 0. After sending, it sets reminder_sent = 1 to prevent duplicate emails.

```
SELECT r.reg_id, r.first_name, s.name,
       COALESCE(r.notification_email, s.email) AS recipient_email,
       e.event_name, e.event_date, e.event_time,
       COALESCE(v.venue_name, e.venue) AS venue_display
FROM registrations r
JOIN events e ON r.event_id = e.event_id
JOIN students s ON r.student_id = s.student_id
LEFT JOIN venues v ON e.venue_id = v.venue_id
WHERE r.reminder_sent = 0
  AND ADDTIME(CAST(e.event_date AS DATETIME), COALESCE(e.event_time, '00:00:00'))
      BETWEEN NOW() AND DATE_ADD(NOW(), INTERVAL 24 HOUR);
```

9. User Flows

Student Registration & Login

```
Visit /register → Fill name, username, email, password
  → OTP sent to email → /verify_email → Verify OTP
  → Account created → /student_login → /dashboard
```

Google OAuth Login

```
Click "Continue with Google" → Google Account Selection
  → /google_callback → OTP sent to Google account email
  → /verify_google_otp → Verify OTP → /dashboard
```

Event Registration

/events → Click "Register" → /register_event/<id>
→ Fill academic details (name, dept, roll no, year, section, phone)
→ Optionally add notification email (OTP-verified via AJAX)
→ Submit → Confirmation email sent → /registration_success
→ Event appears on /my_events with attendance status

Admin Attendance Marking

/admin_events → "Completed" tab → Click "Mark Attendance"
→ /admin_attendance/<event_id> → See all registered students
→ Check present / uncheck absent → Submit
→ Attendance saved → Students see result on /my_events

Sub-Admin Creation

/permissions → Fill username + email + password + check permissions
→ New admin created in DB → Credentials emailed to new admin
→ Sub-admin logs in at /admin_login → Permissions control their actions

Forgot Password (Student or Admin)

/forgot_password → Enter email → OTP sent to email
→ /verify_reset_otp → Verify OTP
→ /reset_password_form → Enter + confirm new password
→ Password updated → Redirect to login

10. Project Structure

```
event_management_project/
│
├── app.py                        # All routes, DB logic, scheduler
├── requirements.txt              # Python dependencies
├── Procfile                      # Gunicorn config for deployment
├── .env                          # Secrets (DB, JWT, Google OAuth, Mail)
│
├── templates/
│   ├── home.html                 # Landing page
│   ├── select_role.html          # Student / Admin selector
│   │
│   ├── register.html             # Student registration form
│   ├── verify_email.html         # OTP verification (registration)
│   ├── student_login.html        # Student login
│   ├── verify_google_otp.html    # OTP verification (Google OAuth)
│   ├── forgot_password.html      # Forgot password (student)
│   ├── verify_reset_otp.html     # OTP verification (student reset)
│   ├── reset_password.html       # New password form (student)
│   │
│   ├── dashboard.html            # Student home dashboard
│   ├── events.html               # Browse upcoming events
│   ├── event_registration_form.html # Register for event
│   ├── event_created.html        # Event creation success
│   ├── registration_success.html # Registration success
│   ├── my_events.html            # Student's registered events + attendance
│   ├── student_profile.html      # Student profile view/edit
│   │
│   ├── admin.html                # Admin login
│   ├── verify_admin_email_otp.html # OTP verification (admin login)
│   ├── admin_forgot_password.html  # Forgot password (admin)
│   ├── verify_admin_reset_otp.html # OTP verification (admin reset)
│   ├── admin_reset_password.html   # New password form (admin)
│   │
│   ├── admin_dashboard.html      # Admin home dashboard (stats)
│   ├── analytics.html            # Analytics charts dashboard
│   ├── admin_events.html         # Upcoming + completed events tabs
│   ├── admin_event.html          # Registrations for one event
│   ├── attendance.html           # Attendance marking page
│   ├── create_event.html         # Create event form
│   ├── edit_event.html           # Edit event form
│   ├── venues.html               # Venue management
│   ├── edit_venue_form.html      # Edit venue form
│   ├── permissions.html          # Sub-admin permissions management
│   ├── rsvp.html                 # Guest RSVP form (public)
│   ├── admin_rsvps.html          # View RSVPs for an event
│   │
│   └── errors/
```

```
│       ├── 400.html  401.html  403.html
│       ├── 404.html  409.html  500.html
│
└── static/
    └── style.css              # Global CSS (glass-card UI)
    └── error.css              # Error page styles
```

11. Challenges & Learning

Database Connection Stability

The MySQL connection would drop silently after periods of inactivity, causing OperationalError: MySQL Connection not available. This was solved by writing a _DB wrapper class with a _ping() method that auto-reconnects using mysql.connector's built-in reconnect logic before every query.

Schema Evolution Without Downtime

Rather than using a migration framework, the app runs ALTER TABLE ... ADD COLUMN IF NOT EXISTS-style statements at startup (wrapped in try/except) to add new columns to existing tables. This allowed adding username, phone, reminder_sent, notification_email, and profile columns to live tables without data loss.

OTP Session Management

Managing multiple concurrent OTP sessions (registration, Google OAuth, email change, phone change, password reset, notification email) in Flask's server-side session required careful key namespacing to prevent one OTP flow from clobbering another.

Scheduler and Flask App Context

The APScheduler background job runs outside of Flask's request context, which means mail.send() would fail without an active app context. The solution was to wrap the email-sending loop in with app.app_context(): inside the scheduler job.

Venue Availability in Real Time

The event creation form uses a JavaScript fetch() call to /available_venues?date=&slot= — a JSON API endpoint — to show which venues are already booked for a given date and time slot (morning/afternoon), without reloading the page.

Project by Rushitha Borra | EventBliss — Event Management System